RustDesk 无人值守远程 部署运行说明书

适用版本：RustDesk 1.5.0（两台机器同版本，`RustDesk.exe --version` 实测输出 `1.5.0`）
适用系统：Windows 10 专业版 19045 及以上
编写日期：2026-11-03
网络前提：两台机器在同一 Tailscale tailnet（`zhangxiaoxian534@`），走 IPv6 直连

本说明书解决一件事：**让第二台电脑一开机就能被远程连上并操控，不需要有任何人坐在那台机器前点"接受"。**

达成这个目标要过三道彼此独立的门——**网络、准入、画面**，任何一道没过都会表现成"连不上"，但病因和修法完全不同。第 1 节讲原理，第 2 节讲正确做法，第 3 节是验收方法，第 4 节是这次真实踩的坑。

---

0. 本机当前状态一览（全部实测，非推断）

项目	控制端（这台）	被控端（对面）
机器名	`WIN-ODUCO8TK731`	`desktop-a1lojch`
tailnet 地址	`100.75.155.82`	`100.93.45.7`
RustDesk ID	**`492126054`**	**`261623966`**
登录账号	Administrator	Administrator
版本	1.5.0	1.5.0（peer_info 回报）
21118（直连端口）	不监听（正常，它是控制端）	**OPEN**
21115 / 21116	—	NOT-ACCEPTING（直连不需要）
3389 / 22	—	OPEN / OPEN
链路延迟	18–19 ms，`pong from desktop-a1lojch ... via [240e:...]:41641`（直连，非中继）	
编码能力	解码 H.265/H.264 走 D3D11VA 硬解	支持 h264/h265/vp8/av1
对面报的显示器		`\\.\DISPLAY1` 1600x900 online scale 1.0，单屏
对面虚拟显示驱动		`idd_impl: amyuni_idd`
本机凭据文件	`AppData\Roaming\RustDesk\config\peers\100.93.45.7.toml`，第一行 `password = []` ⇒ **没记住密码**	

> **ID 归属是这次最容易搞错的一件事。** `492126054` 是**这台机器自己的** ID，`261623966` 是**对面的**。别拿会话标题栏或历史列表里的数字当"自己的 ID"，那是对面的。

自己那台机器的 ID 只有一个可靠拿法：

```
"C:\Program Files\RustDesk\RustDesk.exe" --get-id
```

---

1. 原理：三层模型

RustDesk 远程能不能出画面，是三层各自独立成立的结果。**排障时必须先定位在哪一层，不要笼统说"连不上"。**

第 1 层：网络（TCP 能不能建起来）

判据：`21118` 端口能不能连上。

- 直连模式：控制端直接 TCP 连 `被控端IP:21118`，绕过所有中继服务器。这是最干净的一条，本例走 Tailscale 的 IPv6 直连，18 ms。
- 中继模式：连不上 21118 时才回退到公共中继 `rs-ny.rustdesk.com:21116`，速度受中继带宽限制。

这一层挂了的表现：`Failed to connect to 100.93.45.7:21118: 请稍后再试`。跟密码、跟权限一点关系都没有。

第 2 层：准入（对面让不让你进去）

这一层有**两条并行的门，任何一条过了就进门**：

通道	怎么过	需要人在对面吗
A 人工同意	对面弹出"远程连接请求"，有人点**接受**	**需要**
B 密码	控制端输密码。密码又分**固定密码**（自己设的、永久）和**一次性密码**（对面主界面显示那 6 位，会变）	固定密码**不需要**；一次性密码需要有人看屏幕

所以"无人值守"的定义很清楚：**必须有固定密码，并且对面服务在跑。**只有 A 通道的机器，你重启完它就永远等在弹窗前。

配置项对应关系（对面 RustDesk → 设置 → 安全）：

- **允许密码或点击访问** —— 两条通道都开，这是对的。
- **同时使用两种密码** —— 勾上后，即使有固定密码，一次性密码也要同时正确。**无人值守别勾**，勾了就只能靠人报密码。
- **启用双重认证** —— 需要外部验证器，无人值守场景**保持不勾。**
- **设置固定密码** —— 这是**唯一有效的设密码入口。**

固定密码存在哪：对面 `RustDesk.toml` 的 `password` 字段，是**加盐哈希**（`01` 开头）。加盐意味着同一个密码每次写入得到的哈希都不一样 —— **哈希不能用来证明"现在存的是哪个密码"**，只能证明"变了"。

第 3 层：画面（帧有没有真的过来）

第 2 层过了，界面会显示"已连接"，但**"已连接"不等于有画面**。RustDesk 的会话里，控制通道、音频、剪贴板、视频是各自独立的流。视频起不来时你看到的是**黑屏 + "等待画面传输"**，而音频和剪贴板可能是通的。

这一层挂了的表现：黑屏、"等待画面传输"，然后对面把 TCP 拆了 → 控制端报 `远程主机强迫关闭了一个现有的连接。(os error 10054)`。

进程模型：为什么"能连但随时掉"

Windows 上 RustDesk 有两个东西在跑，职责完全不同：

	装的服务（session 0）	界面进程（session 1，托盘那个）
能不能抓锁屏/登录界面	**能**（安全桌面）	**不能**
注销、切换用户、RDP 抢会话之后	还在	**会被干掉**
没人登录时	在跑	不存在

这次真正的根因就在这张表上。 对面曾经执行过 `--remove`，服务被卸掉了，`21118` 改由 session 1 的界面进程扛着。那种状态下：能连、能过密码、但会话一被注销或 RDP 抢走，界面进程没了 → 它持有的 TCP 被 RST → 就是你看到的 10054，而且画面一帧都发不出来（因为那个进程当时抓不到可用桌面）。

结论：`21118` 必须由服务持有，托盘不能退，`RustDesk 服务` 必须是 Running + Automatic。

---

2. 正确做法（照抄即可）

2.1 对面（被控端）

第 1 步 • 装服务并确认（管理员 PowerShell）：

```
& "C:\Program Files\RustDesk\RustDesk.exe" --install-service
Get-Service RustDesk | Select-Object Status, StartType
```

结果必须是 `Running` + `Automatic`。

> ⚠ **这一步在 Git-Bash 环境里会挂死**，见第 4 节坑 12。必须在真正的 PowerShell / cmd 里跑，别在带 Git PATH 的 shell 里跑。

第 2 步 • 设固定密码：RustDesk 主界面 → 设置 → 安全 → **设置固定密码**。不要用命令行（坑 8、坑 9）。

第 3 步 • 准入开关：`允许密码或点击访问` 开；`同时使用两种密码` 关；`启用双重认证` 关。

第 4 步 • 托盘保留：右下角 RustDesk 托盘图标**不要退出**。退出只是关掉界面进程，但会削弱功能，且没有任何收益。

第 5 步 • 电源设置（管理员 PowerShell）——这是"屏幕关掉但机器不断连"的关键：

```
powercfg /change monitor-timeout-ac 1
powercfg /change monitor-timeout-dc 1
powercfg /change standby-timeout-ac 0
powercfg /change standby-timeout-dc 0
powercfg /change disk-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /setacvalueindex SCHEME_CURRENT SUB_SLEEP HIBERNATEIDLE 0
```

读回验证（读出来的单位是**秒**）：

```
powercfg /q SCHEME_CURRENT SUB_VIDEO VIDEOIDLE | Select-String '0x'
```

看到 `Current Access Setting Index Value: 0x0000003c` = 60 秒，正确。

第 6 步 • 关掉远程改配置的口子：设置 → 安全 → `允许远程修改配置`（`allow-remote-config-modification`）**关**。开着它，任何拿到密码的人都能改对面那台机器的 RustDesk 配置。

2.2 控制端（这台）

1. ID 框填**对面的 IP**（`100.93.45.7`）或对面的 ID（`261 623 966`）。填自己那台的 ID 是连自己，会失败。
2. 输密码时**勾上"记住密码"**。不勾的话凭据文件里就是 `password = []`，每次重连都要重新问 —— 这次就白踩了一遍。
3. 电源：同样把显示器设 1 分钟、睡眠设 0。

2.3 顺手把 SSH 通道建起来（可选但强烈推荐）

RustDesk 是"人在才好用"的通道；SSH 才是能让你或脚本半夜自己进去干活的通道。对面 `22` 已经 OPEN，只差授权公钥。

坑：Windows OpenSSH 对 Administrators 组的用户不读 `~\.ssh\authorized_keys`，只读 `C:\ProgramData\ssh\administrators_authorized_keys`，而且这个文件的 ACL 必须严格——只要继承来了普通用户权限，sshd 会**直接忽略整个文件**，不报错，只是拒绝你。

正确写法（对面，管理员 PowerShell，整块粘贴）：

```
$pub = 'ssh-rsa AAAA...你的公钥一行... user@host'
$f = 'C:\ProgramData\ssh\administrators_authorized_keys'
New-Item -ItemType Directory -Force -Path 'C:\ProgramData\ssh' | Out-Null
$old = @(); if (Test-Path $f) { $old = @(Get-Content $f | Where-Object { $_.Trim() -ne '' }) }
$keep = @($old | Where-Object { $_ -ne $pub }); $keep += $pub
Set-Content -Path $f -Value $keep -Encoding ASCII
icacls $f /inheritance:r /grant:r '*S-1-5-18:F' '*S-1-5-32-544:F'
(Get-Service sshd).Status
```

`icacls` 那两行是 `SYSTEM:(F)` 和 `Administrators:(F)`，别的都不能有。

---

3. 怎么判断"到底成没成"：唯一有效的验收

端口 OPEN、服务 Running、能连进一个已登录的桌面 —— 这三条全绿都不能证明"无人值守"成立。 它们只证明"有人登录之后能连"。

真正的判据只有一个：**让对面处于"没有用户会话"的状态，还能不能连上并看到画面。**

两种做法，第二种更快：

- **彻底版**：对面重启，重启后**不登录任何人、不开 RDP**，控制端连过去。
- **快速探针版**：对面已登录状态下按 **Win+L** 锁屏，控制端连过去。**能看到锁屏界面、并且能把密码打进去解锁**，就说明抓画面的是 session 0 的服务（界面进程抓不到安全桌面，只会给黑屏）。

本次实测（快速探针版，2026-11-03 08:51:57）：

```
08:51:57.389  handle_peer_info                              ← 第 2 层过了
08:51:57.597  recved audio format, sample rate=48000       ← 0.2 秒，正常
08:51:57.991  new video handler for display #0, format: H265, luid: 70686   ← ★ 第 3 层过了
08:51:58.026  create H265 decoder success
08:52:03.240  Set fps to 30
之后 6 分钟 0 次 Connection closed
```

同期端口采样（08:45:18 → 08:57:15，每 5 秒一次，共 142 次）：`21118` **OPEN 142 次、DOWN 0 次。**

锁屏状态下能出画面并解锁 ⇒ 无人值守成立。

---

4. 这次真实踩的坑（每一条都付过代价）

坑 1：`powercfg /change` 那三个 timeout 别名的单位是分钟，不是秒

`powercfg /change monitor-timeout-ac 60` 的意思是 **60 分钟 = 3600 秒**，不是"60 秒灭屏"。要 60 秒灭屏必须写 `1`。

写 60 之后读回是 `0x00000e10`（3600），写 1 之后读回 `0x0000003c`（60）—— 这就是发现过程。**凡是改完电源设置，必须用 `powercfg /q` 读回那个 0x 值自己换算一遍**，别信参数名。

坑 2：把"灭屏"当成"断远程"

灭屏不会断远程连接，只有睡眠/休眠会。 显示器超时是安全的，`standby` / `hibernate` 才是杀手。所以正确组合是"显示器 1 分钟 + 睡眠 0 + 休眠关"，不是"永远不关屏幕"。

坑 3：把控制端误判成"正在被人控制"

`powercfg /requests` 在**这台**机器上显示 `DISPLAY` 被 `RustDesk.exe` 持有。第一反应是"我被远程控制着"——**错的。**控制端**自己开着的远程会话窗口**就会持 `ES_DISPLAY_REQUIRED`，这是客户端行为，不是被人控。

判据：这台机器**没有任何入站监听**（`21118` 不监听），且 `powercfg /requests` 里持有者是 GUI 进程。

坑 4：会话窗口的"不许灭屏"没有开关可以关

RustDesk 客户端窗口只要开着，就天然持 `ES_DISPLAY_REQUIRED`，**配置项里没有任何开关能关掉它。**想让屏幕真的黑下去，只有三条路：关掉会话窗口、用 DDC/CI 直接命令显示器背光（Monitorian / Twinkle Tray）、或者接受"面板亮着但没人看"。

坑 5：拿"端口通不通"当"连没连上"的判据

10054 发生的那一整段时间里，`21118` **一直是 OPEN。**反过来说，**事后测到 OPEN 也不能证明当时没断。**要判"当时断没断"只能看日志里那条 `Connection closed` 的时间戳。

坑 6：`os error 10054` 不是"连不上"，是"连上了被对面拆掉"

10054 = `WSAECONNRESET`，是**对面**给这条**已建立**的 TCP 发了 RST。Windows 上，进程退出时如果它的 socket 接收缓冲里还有没读完的数据，就会发 RST 而不是 FIN。所以 10054 的实际含义是：**对面的 RustDesk 服务进程没了/被换了。**

看到 10054 还去查密码、查防火墙，方向就错了。

坑 7：只看界面写"已连接"就当成功

界面上"成功 / 已连接，等待画面传输..."是真的连上了，但那一层只到第 2 层。**唯一算数的日志行是** `new video handler for display #0`，它代表第一帧视频真的到了。缺这行 = 对面一帧画面都没发。

失败那次（08:37）和成功那次（08:51）的对比，就差在这一行：

日志标记	06:09 成功	08:37 失败	08:51 成功
`handle_peer_info`	有	有	有
`recved audio format`	0.24 秒后	**9.6 秒后**（迟到）	0.21 秒后
`cpu usage synced`	有	**完全没有**	有
`new video handler`	有	**没有**	有
`Set fps to 30`	有	没有	有
结局	稳定控制	49 秒后 10054	稳定

`cpu usage synced` 是对面服务器线程在正常跑心跳的旁证；它整段消失 + 音频迟到 10 倍，说明对面的服务当时已经处于半瘫状态。

坑 8：`RustDesk.exe --password "xxx"` 这个"有效写法"是假的

网上/别的模型给的这条命令，在 1.5.0 上**不存在。**实测：在 `librustdesk.dll`（40,680,304 字节）和 `RustDesk.exe`（369,520 字节）里搜 `--password`，**零命中**；而 `--get-id / --set-id / --option / --config / --connect / --install-service / --remove / --assign / --deploy / --check-hwcodec-config` 都能在 DLL 里搜到。

结论：设固定密码只有 UI 那一条路。 任何声称命令行能设密码的稿子，先让他 `--version` 并当场演示。

坑 9：`--option permanent-password "xxx"` 同样无效，而且会留垃圾

DLL 里 `permanent-password` 出现 8 次，但绝大多数是 `disable-change-permanent-password`、`use-permanent-password`、`temporary-password...` 这类**别的键名**里的子串。拿它当"设密码"的开关写进配置，结果只是在 `[options]` 里多一个没人读的死键，密码纹丝不动。

判据（这次学到的方法）：找到一个字符串常量 ≠ 找到一条命令。必须去二进制里找**读这个键的代码**，或者直接看改完之后的结果字段变没变。

坑 10：加盐哈希不能用来证明"密码是哪个"

对面 `RustDesk.toml` 里 `password` 字段是 `01` 开头的加盐哈希。**同一个密码，每次写入得到的哈希都不一样**（盐是随机的）。所以"我写了 A 密码，哈希从 X 变成 Y"只证明"发生过一次写入"，不证明"存进去的是 A"。

要验证密码对不对，只有一个办法：**用它连一次。**

坑 11：没勾"记住密码"，于是每次重连都要手输

凭据文件 `peers\100.93.45.7.toml` 第一行是 `password = []`（空数组）。这不是 bug，是没勾。勾上以后这一行会变成非空 —— 顺手拿这个当"记住密码成功"的验收判据。

坑 12：`--install-service` 在 Git-Bash 环境里会挂死

对面那次挂死的真因：`D:\Git\usr\bin\timeout.exe` **抢在** `C:\Windows\System32\timeout.exe` **前面。** RustDesk 的安装脚本内部调 `timeout`，拿到的是 Git 的 coreutils 版本，参数语法完全不同：

```
/usr/bin/timeout: invalid time interval '/t'     ← 退出码 125
```

脚本等不到它返回就一直挂着。**症状是"命令没反应、也没报错"**，很容易被误判成 RustDesk 的问题。

修法：装服务用干净的 PowerShell/cmd，别用带 Git PATH 的 shell。

坑 13：PowerShell 5.1 按 ANSI 读无 BOM 的 `.ps1`，中文标识符会被毁

给对面写的脚本一律 **纯 ASCII**，或者存成带 BOM 的 UTF-8。这次两份脚本都是 ASCII-only 并逐字节验过 0 个非 ASCII 字符。

顺带一条：在 Bash 工具里跑 `powershell -Command "...$var..."`，`$var` 会被 shell 吃掉，报一堆"foreach 缺少变量名称"。**脚本里有变量就写成 `.ps1` 用 `-File` 跑。**

坑 14：`Set-Clipboard` 会因 RustDesk 的剪贴板监听而失败

RustDesk 会话开着时它自己有一个剪贴板监听线程，`Set-Clipboard` 会抛 `ExternalException: 打开剪贴板失败`。绕法：把内容写进文件，用 `clip.exe` 灌：

```
cmd /c "clip < F:\_scratch\host-key-block.txt"
```

坑 15：明文密码的落点

固定密码一旦出现在聊天记录或剪贴板里，就等于留了底。收尾三件事：用完清空剪贴板（`Set-Clipboard -Value ''` 或 `clip` 灌空格）、通过 UI 换一个新固定密码、别把密码写进任何 `.ps1` 文件（公钥可以写文件，

私钥和密码不行）。

---

5. 故障速查表

屏幕上的话	哪一层	先查什么
`Failed to connect to IP:21118: 请稍后再试`	第 1 层 网络	对面 `21118` 通不通；对面服务在不在；对面是不是退托盘/注销了
弹"需要密码"	第 2 层 准入	固定密码设了没；`同时使用两种密码` 是不是误勾了
弹"等待对方接受"	第 2 层 准入	对面没设固定密码，只剩人工通道
黑屏 + `已连接，等待画面传输…`	第 3 层 画面	日志里有没有 `new video handler`；对面是不是抓不到桌面
`远程主机强迫关闭了一个现有的连接 (os error 10054)`	第 3 层（对面进程被换）	对面服务是不是挂了/被重启；是不是 RDP 抢了会话；托盘是不是退了
连上后能看画面但**看不到锁屏**	进程模型	`21118` 被 session 1 的界面进程持有，不是服务 ⇒ 重装服务
每次重连都要重新输密码	凭据	`peers\*.toml` 里 `password = []` ⇒ 勾"记住密码"
屏幕关不掉	电源	控制端会话窗口持 `ES_DISPLAY_REQUIRED`，无开关；改走 DDC/CI

---

6. 待办（写这份说明书时仍未闭合的）

项	状态	影响
对面装 SSH 公钥	块已生成并放进剪贴板，**还没执行**	装完才能不进 RustDesk 也能操作对面
勾"记住密码"	未做（`password = []`）	每次重连多一次手输
关 `allow-remote-config-modification`	未做	远程可改对面配置
轮换固定密码	未做	当前那个密码已在两条对话记录里留底；本文**不写**密码明文
彻底版验收（重启后不登录再连）	未做	快速探针版已通过，彻底版更硬

---

7. 一句话记住这套东西

> **网络层看端口，准入层看密码，画面层看 `new video handler`；能连进已登录的桌面不算无人值守，能连进锁屏才算。**